

TPG4190 Seismic data acquisition and processing

Lecture 9: Processing

B. Arntsen

NTNU

Department of Geoscience and petroleum
borge.arntsen@ntnu.no

Trondheim fall 2025

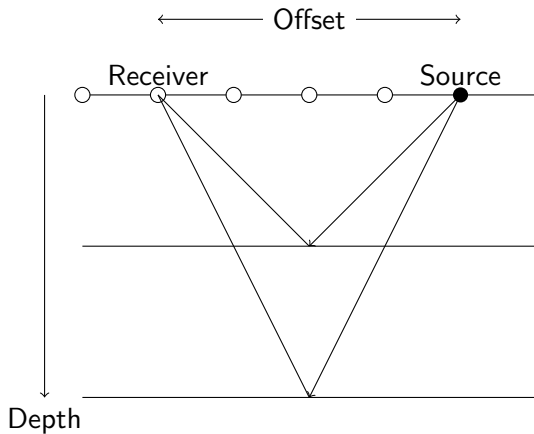
Overview

- ▶ Filters
- ▶ Basic processing sequence
- ▶ Basic+ processing sequence
- ▶ Basic++ processing sequence

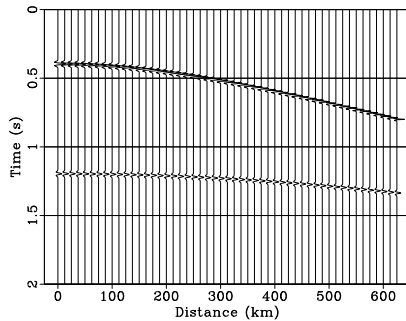
Basic processing sequence

1. Input data
2. Velocity analysis
3. NMO + Stack
4. Output result

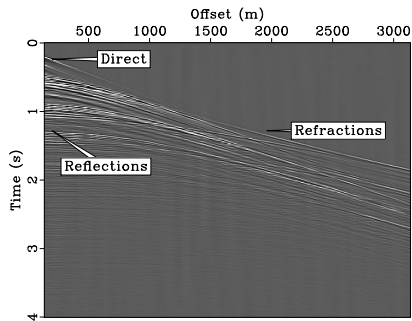
Seismic marine data acquisition



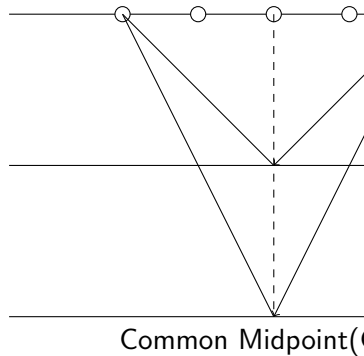
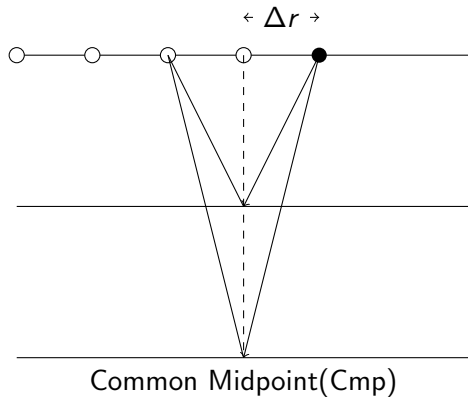
Schematic shot record



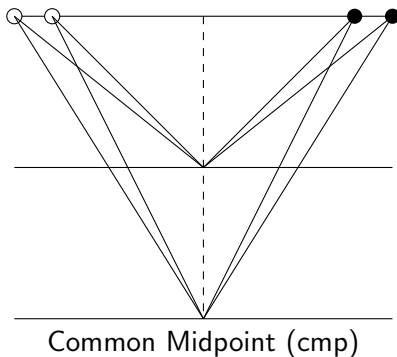
Real shot record



The cmp method



Common midpoint (cmp) gather



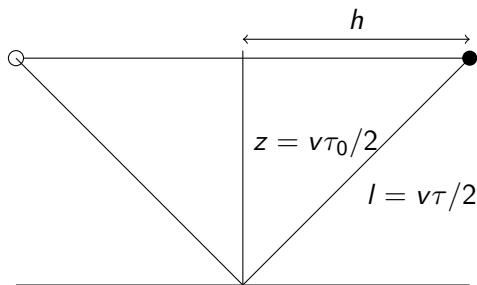
Midpoint and Offset Coordinates

$$x_m = \frac{s + r}{2}, \quad (1)$$

$$h = \frac{s - r}{2}, \quad (2)$$

- ▶ x_m : Midpoint coordinate
- ▶ h : Offset
- ▶ s : Source coordinate
- ▶ r : Receiver coordinate

NMO and Stack



The travelttime $\tau(h)$ is:

$$l^2 = z^2 + h^2 \quad (3)$$

which gives by inserting $v\tau/2$ for l and $v\tau_0/2$ for z

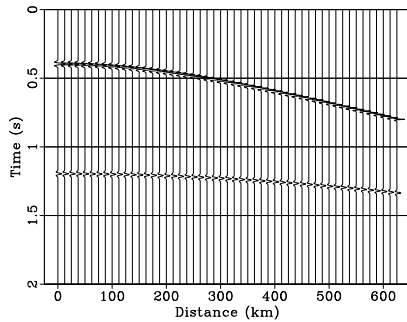
$$\tau(h) = \sqrt{t_0^2 + 4h^2/v^2}. \quad (4)$$

NMO and Stack

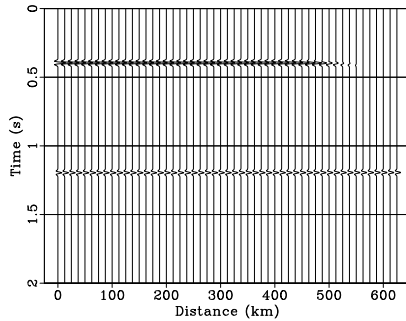
Nmo-correction:

$$\Delta\tau = \tau(h) - \tau_0, \quad (5)$$

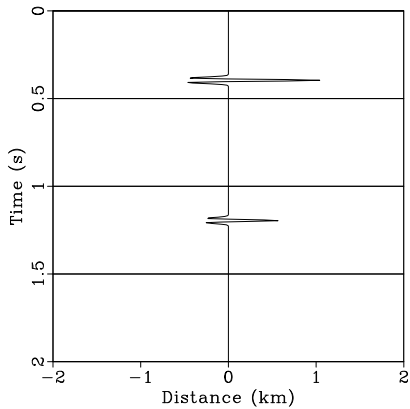
Cmp



Nmo



Stack



NMO and Stack

Average velocity v_{rms} defined by

$$v_{rms}^2(t_0) = \frac{1}{t_0} \int_0^{t_0} v^2(t) dt \quad (6)$$

$v(t)$: Interval velocity. The travelttime equation (4) then becomes

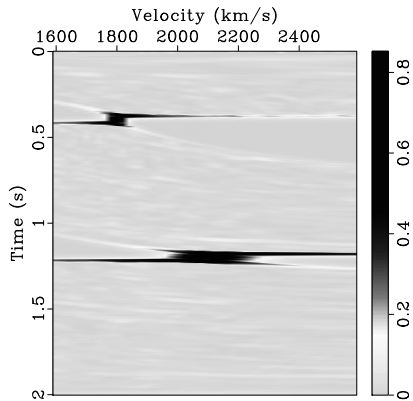
$$\tau(h) = \sqrt{t_0^2 + 4h^2/v_{rms}^2(t_0)}. \quad (7)$$

Velocity analysis

Estimate v_{rms} from equation (4)

1. Make a guess of v
2. Perform Nmo correction for all t_0 with using guess of v
3. Make a stack trace
4. Perform above steps for a range of guesses of v
5. Plot all stack traces in a velocity spectrum

Velocity spectrum



Semblance

Stack is usually replaced with semblance to get better velocity spectra

$$S(t) = \frac{\int_0^{h_{max}} p^2(t, h)}{\int dt_0^T \int_0^{h_{max}} p^2(t, h)} \quad (8)$$

- ▶ $p(t, h)$: data
- ▶ h_{max} : maximum offset.

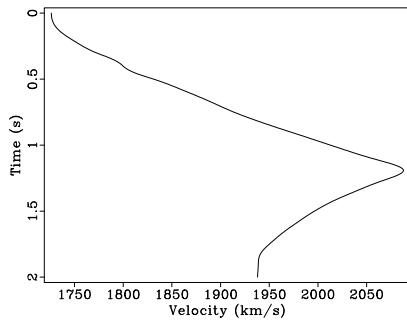
This equation can not be used directly. Why?

Semblance

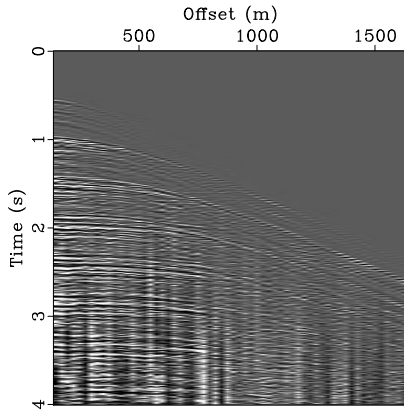
$$S_k = \frac{\sum_{i=0}^{N_h} p_{k,i}^2}{\sum_{k=0}^{N_t} \sum_{i=0}^{N_h} p_{k,i}^2} \quad (9)$$

- ▶ $S_k = S(t = k\Delta t), k = 0, \dots, N_t$
- ▶ $p_{k,i} = p(t = k\Delta t, h = i\Delta h), i = 0, \dots, N_h.$
- ▶ Δt time sampling interval,
- ▶ Δh distance between offsets
- ▶ N_t, N_h : No of time samples and No of offsets

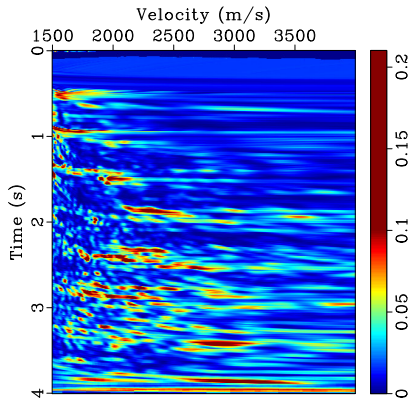
Velocity analysis



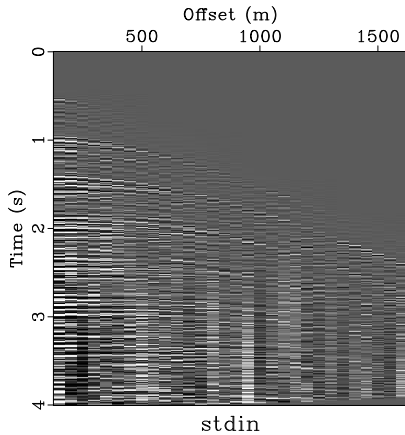
Velocity analysis



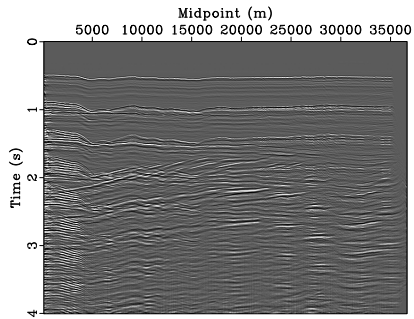
Velocity analysis



Velocity analysis



Velocity analysis



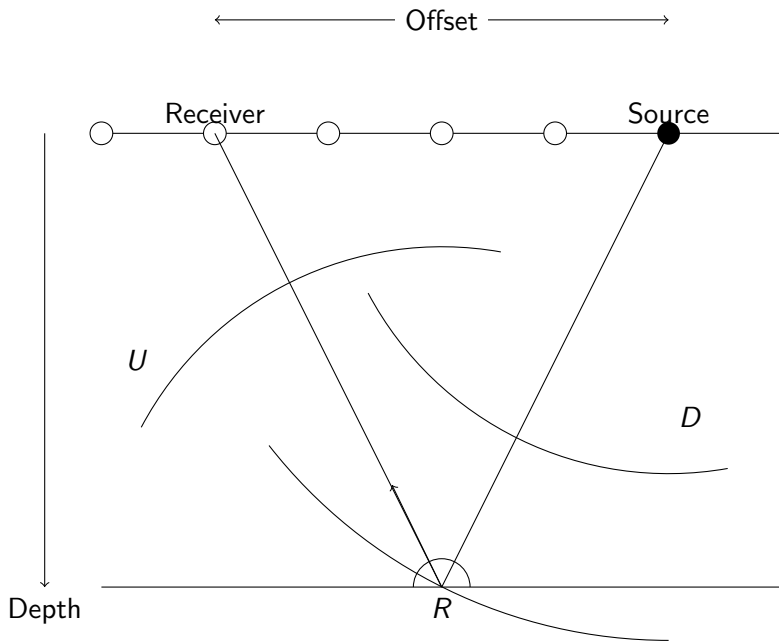
Summary

- ▶ Seismic data acquisition
- ▶ CMP method
- ▶ NMO-correction
- ▶ Velocity analysis

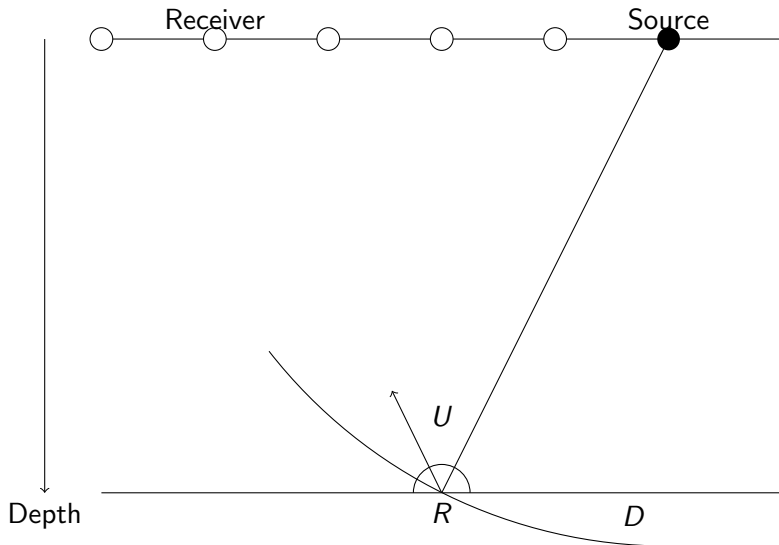
Overview

- ▶ Migration principles
- ▶ Kirchhoff Time migration

Migration principles



Migration principles



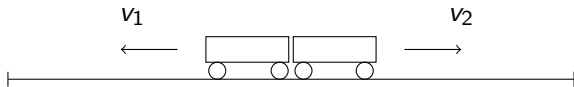
Migration principles

Make an image by cross-correlating the downgoing and upgoing waves

$$R(\mathbf{x}) = \int dt U(\mathbf{x}, t) D(\mathbf{x}, t), \quad (10)$$

where R is the reflectivity and $\mathbf{x} = (x, y, z)$ denotes a position in space, while t is the time.

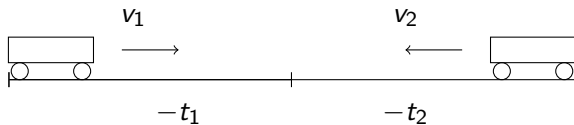
Time reversal 1



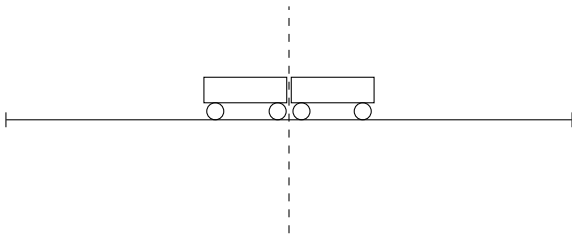
Time reversal 2



Time reversal 3



Time reversal



Seismic imaging

Forward modeling

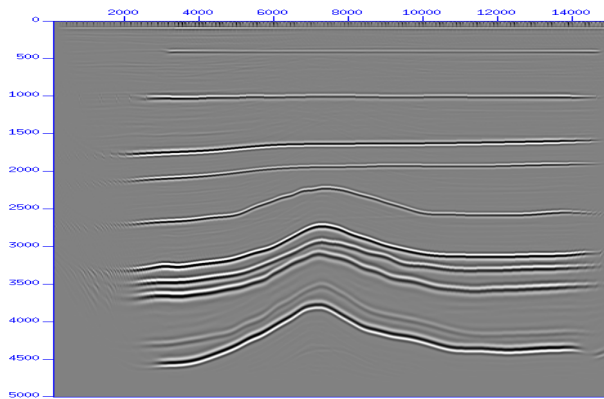
Time reverse modeling

Backward modeling

Cross correlation

Imaging

Complete image



Kirchhoff migration

Simplest approach for up- and downgoing waves

$$\begin{aligned}D(\mathbf{x}, t) &= A\delta(t - \tau_s), \\U(\mathbf{x}, t) &= BP(t + \tau_r),\end{aligned}\tag{11}$$

where

- ▶ $\mathbf{x}$: reflection point,
- ▶ A and B : amplitude factors
- ▶ P : Data at the surface
- ▶ τ_s travelttime from the source to the reflection point
- ▶ τ_r travelttime from the receiver to the reflection point

Kirchhoff migration

Using equation (11) and equation (10), I get

$$R(\mathbf{x}) = \int dt A \delta(t - \tau_s) B P(t + \tau_r), \quad (12)$$

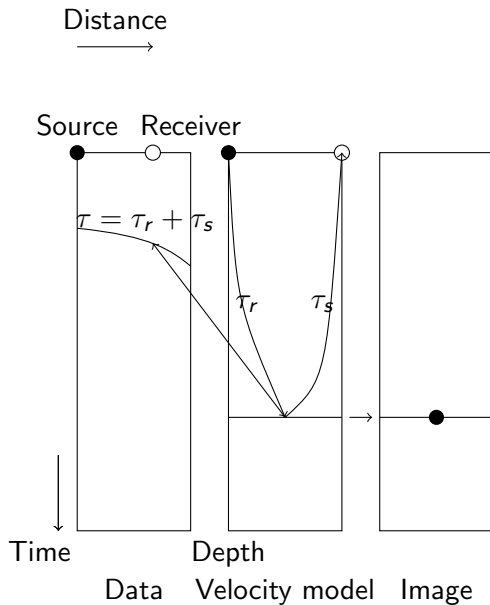
which after integration gives

$$R(\mathbf{x}) = ABP(\tau_s + \tau_r). \quad (13)$$

If we disregard the amplitude factor AB the image is simply equal to the amplitude of the recorded data at a time equal to

$$\tau = \tau_r + \tau_s.$$

Kirchhoff migration



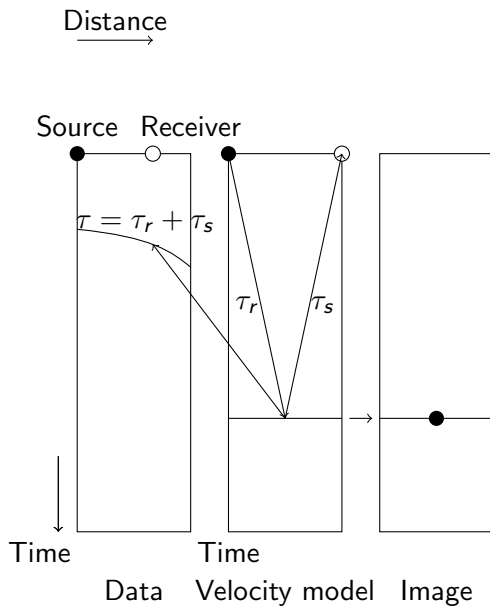
Kirchhoff migration

Many source-receiver pairs will contribute to the same imaging point, so that equation (13) becomes

$$R(\mathbf{x}) = \sum_{r,s} P(\tau_s + \tau_s), \quad (14)$$

where we have neglected the amplitude factors and the summation is over all source-receiver pairs contributing to the image point.

Kirchhoff migration



Kirchhoff migration

Assume velocity $c(\mathbf{x})$ is depth dependent only, $c = c(z)$, then

$$\begin{aligned}\tau_s &= \sqrt{\frac{(x - x_s)^2 + (y - y_s)^2}{c^2} + \tau_0^2}, \\ \tau_r &= \sqrt{\frac{(x - x_r)^2 + (y - y_r)^2}{c^2} + \tau_0^2},\end{aligned}\tag{15}$$

- ▶ $(x_r, y_r), (x_s, y_s)$: Source, receiver positions
- ▶ $\tau_0 = z/c$: vertical traveltimes and
- ▶ c : velocity.

Kirchhoff migration

We then get for the total travelttime τ

$$\tau = \sqrt{\frac{(x - x_s)^2 + (y - y_s)^2}{c^2}} + \tau_0 + \sqrt{\frac{(x - x_r)^2 + (y - y_r)^2}{c^2}} + \tau_0. \quad (16)$$

The image R is then

$$R(x, y, \tau_0) = \sum_{r,s} P(\tau_s + \tau_s), \quad (17)$$

Zero-offset simple migration

We get for zero-offset (stack) data (2D)

$$\tau = 2\sqrt{\frac{(x - x_s)^2}{c^2} + \tau_0^2}. \quad (18)$$

Zero-offset simple migration

Migration of zero-offset section can then be done as:

- ▶ Chose a point x_s, τ_0 on the section.
- ▶ Loop over all x values, compute τ and sum all $P(\tau)$
(Summing over an hyperbola with center at x_s, τ_0 .)
- ▶ Put the sum at location x_s, τ_0 in the output.
- ▶ Repeat for all possible points x_s, τ_0 .

Basic+ processing sequence

1. Input data
2. Velocity analysis
3. NMO + Stack
4. Zero-offset (poststack) migration,
5. Output result

Nobody with full possession of their faculties uses a processing sequence like this today, except for initial QC.

Basic++ processing sequence (time)

1. Input data
2. preprocessing (designature, debubble, etc..)
3. Multiple removal
4. Initial prestack migration
5. Velocity analysis on demigrated data.
6. Final prestack migration
7. Multiple removal
8. Stack
9. Output result